CSA02 – ASSIGNMENT 1

Grama Jyothi Rural Electricity Co-operative
C Programming: Complete Answers with C Code

Name: BAVADARANI S

Register Number: 1912260562

GitHub Repository Link: https://github.com/bavadarini28/ CSA0209-1912260562-BAVADARANI-S-ASSIGNMENT-1.git

Scenario and Design

Grama Jyothi processes monthly electricity readings. The program must calculate consumption correctly, handle a 16-bit meter rollover from 65535 to 0, use a single definition for tariff constants, validate input, and avoid floating-point errors in currency. The example tariff used in this report is: first 100 units free; units 101–200 at Rs 2.25 per unit; units above 200 at Rs 4.50 per unit. Since the question does not specify a fixed charge, this implementation assumes a fixed charge of Rs 30.00. Change the single constant if the co-operative specifies another amount.

Data types: uint16_t for meter readings, uint32_t for consumption, and int64_t for money in paise. Using integer paise means Rs 2.25 is stored as 225 paise and Rs 4.50 as 450 paise.

TASK 1 – Header Files and Conditional Compilation (3 Marks)

Create two files in Task01: billing.h and billing.c. The header holds constants and prototypes.

File: Task01/billing.h

```
#ifndef BILLING_H
#define BILLING_H

#include <stdint.h>

#define FREE_UNITS 100u
#define SECOND_SLAB_LIMIT 200u
#define SECOND_SLAB_PAISE 225u
#define THIRD_SLAB_PAISE 450u
#define FIXED_CHARGE_PAISE 3000LL

uint32_t calculate_units(uint16_t previous, uint16_t current);
int64_t calculate_bill_paise(uint32_t units);
void print_money(int64_t amount_paise);

#endif
```

File: Task01/billing.c

```
#include <stdio.h>
#include "billing.h"

uint32_t calculate_units(uint16_t previous, uint16_t current)
{
    /* Unsigned 16-bit subtraction handles meter rollover modulo 65536. */
    return (uint16_t)(current - previous);
}

int64_t calculate_bill_paise(uint32_t units)
{
    int64_t energy = 0;

    if (units > FREE_UNITS) {
        uint32_t second = units < SECOND_SLAB_LIMIT
                        ? units - FREE_UNITS
                        : SECOND_SLAB_LIMIT - FREE_UNITS;
        energy += (int64_t)second * SECOND_SLAB_PAISE;
    }
    if (units > SECOND_SLAB_LIMIT) {
        energy += (int64_t)(units - SECOND_SLAB_LIMIT) * THIRD_SLAB_PAISE;
    }

#ifdef DEBUG
    printf("[DEBUG] units=%u, energy_paise=%lld\n",
           units, (long long)energy);
#endif
    return energy + FIXED_CHARGE_PAISE;
}

void print_money(int64_t amount_paise)
{
    printf("Rs %lld.%02lld\n",
           (long long)(amount_paise / 100),
           (long long)(amount_paise % 100));
}

int main(void)
{
    unsigned int prev_input, curr_input;
    uint16_t previous, current;
    uint32_t units;
    int64_t bill;

    printf("Enter previous reading (0-65535): ");
    if (scanf("%u", &prev_input) != 1 || prev_input > 65535u) {
        fprintf(stderr, "Invalid previous reading.\\n");
        return 1;
    }
    printf("Enter current reading (0-65535): ");
    if (scanf("%u", &curr_input) != 1 || curr_input > 65535u) {
        fprintf(stderr, "Invalid current reading.\\n");
        return 1;
    }

    previous = (uint16_t)prev_input;
    current = (uint16_t)curr_input;
    units = calculate_units(previous, current);
    bill = calculate_bill_paise(units);

    printf("Units consumed: %u\\n", units);
    printf("Bill: ");
    print_money(bill);
    return 0;
}
```

Explanation

A header file (.h) stores declarations shared by source files. main() controls input and output; helper functions calculate units, calculate the bill, and print money.

The include guard (#ifndef, #define, #endif) prevents the same header contents from being processed more than once in a translation unit.

```
#include "billing.h" searches the local/project include path first. #include <stdio.h> is used for standard/system headers.
```

Compile normally with gcc -std=c11 -Wall -Wextra -pedantic billing.c -o billing. Enable debug output with gcc -std=c11 -Wall -Wextra -pedantic -DDEBUG billing.c -o billing_debug.

With #ifdef DEBUG, debug statements are removed from a normal build by preprocessing. A run-time if (debug) check remains in the executable and adds a condition check whenever reached; it also retains the debug instructions and format string.

Complexity: calculating one bill is O(1) time and O(1) extra space. For n bills, total processing is O(n).

TASK 2 – Read and Print Data Correctly (4 Marks)

Example C program with input validation, rollover handling, and correctly formatted currency:

File: Task02/billing_input.c

```
#include <stdio.h>
#include <stdint.h>

#define FREE_UNITS 100u
#define SECOND_SLAB_LIMIT 200u
#define SECOND_SLAB_PAISE 225u
#define THIRD_SLAB_PAISE 450u
#define FIXED_CHARGE_PAISE 3000LL

static int64_t bill_for_units(uint32_t units)
{
    int64_t energy = 0;
    if (units > FREE_UNITS) {
        uint32_t second = units < SECOND_SLAB_LIMIT
                        ? units - FREE_UNITS
                        : SECOND_SLAB_LIMIT - FREE_UNITS;
        energy += (int64_t)second * SECOND_SLAB_PAISE;
    }
    if (units > SECOND_SLAB_LIMIT) {
        energy += (int64_t)(units - SECOND_SLAB_LIMIT) * THIRD_SLAB_PAISE;
    }
    return FIXED_CHARGE_PAISE + energy;
}

static void print_money(int64_t paise)
{
    printf("Rs %lld.%02lld\\n",
           (long long)(paise / 100), (long long)(paise % 100));
}

int main(void)
{
    unsigned int prev, curr;
    uint32_t units;
    int64_t bill;

    printf("Enter previous reading (0-65535): ");
    if (scanf("%u", &prev) != 1 || prev > 65535u) {
        fprintf(stderr, "Invalid previous reading.\\n");
        return 1;
    }
    printf("Enter current reading (0-65535): ");
    if (scanf("%u", &curr) != 1 || curr > 65535u) {
        fprintf(stderr, "Invalid current reading.\\n");
        return 1;
    }

    units = (uint16_t)((uint16_t)curr - (uint16_t)prev);
    bill = bill_for_units(units);

    printf("Units consumed: %u\\n", units);
    printf("Fixed charge: Rs 30.00\\n");
    printf("Total bill: ");
    print_money(bill);
    return 0;
}
```

Format specifiers

A wrong format specifier can cause incorrect output or undefined behaviour. For example, passing a long to printf with %d is a type mismatch; use %ld for long. scanf returns the number of successful assignments, so check it before using input. For stronger recovery from invalid lines, use fgets() to read a bounded line and strtol() to parse and validate the whole token. Reading a line of length L takes O(L) time and O(B) buffer space for a buffer of B characters. fgets/strtol makes it easier to discard an invalid line and prompt again; scanf can leave unwanted characters in the input stream unless they are consumed.

Sample test

Input: previous reading = 1000, current reading = 1250

Output: Units consumed: 250; fixed charge: Rs 30.00; total bill: Rs 480.00

Calculation: 100 free units + 100 × Rs 2.25 + 50 × Rs 4.50 + Rs 30.00 = Rs 480.00.

TASK 3 – Type Conversion and Precision (4 Marks)

File: Task03/precision.c

```
#include <stdio.h>
#include <limits.h>

int main(void)
{
    int a = 5, b = 2;
    int units = 150;
    long large = (long)INT_MAX + 100L;

    printf("5 / 2 = %d\\n", a / b);
    printf("150 / 100 = %d\\n", units / 100);
    printf("0.1f + 0.2f = %.9f\\n", 0.1f + 0.2f);
    printf("Large long value = %ld\\n", large);
    printf("Casting large long to int is not safe when out of range.\\n");
    return 0;
}
```

Unexpected expressions and explanations

5 / 2 prints 2, not 2.5, because both operands are integers and integer division discards the fractional part.

150 / 100 prints 1, not 1.5. Convert to floating point before division if a fractional result is truly needed; billing slabs should instead use integer comparisons and integer paise.

0.1f + 0.2f may print approximately 0.300000012 because many decimal fractions cannot be represented exactly in binary floating point.

Converting a long value outside the range of int to int is implementation-defined in C. A cast does not make the value fit safely; choose a type that can hold the value.

Rounding and complexity

Use integer paise for currency: Rs 2.25 = 225 paise and Rs 4.50 = 450 paise. If a tariff ever produces fractional paise, define a rule explicitly, such as nearest paise with exact half-paise values rounded upward. Do not round inconsistently at different steps.

A float-based solution rounds the final result only after floating-point calculations. Its error may accumulate with the number of bills n; a rough bound for independent per-bill rounding of at most 0.5 paise is 0.5n paise in aggregate, while floating-point representation error depends on the values and operations. Integer paise is exact within the integer type's range. Both approaches use O(1) time and space per bill, but integer arithmetic is more reliable for money.

TASK 4 – Trace the Compilation Process (4 Marks)

File: Task04/billing.c

```
#include <stdio.h>

int main(void)
{
    unsigned int units = 250;
    long long bill_paise = 3000LL + 100LL * 225LL + 50LL * 450LL;

    printf("Units: %u\\n", units);
    printf("Bill: Rs %lld.%02lld\\n", bill_paise / 100, bill_paise % 100);
    return 0;
}
```

GCC commands

gcc --version
gcc -std=c11 -Wall -Wextra -pedantic -E billing.c -o billing.i
gcc -std=c11 -Wall -Wextra -pedantic -S billing.c -o billing.s
gcc -std=c11 -Wall -Wextra -pedantic -c billing.c -o billing.o
gcc billing.o -o billing
./billing

gcc -E: preprocessor output. Header inclusions and macros are expanded; preprocessing directives are handled.

gcc -S: compiler output in assembly language, billing.s.

gcc -c: creates object code, billing.o, without linking an executable.

gcc billing.o -o billing: links the object file and libraries to create the executable.

Compile-time error: a syntax error such as a missing semicolon. Link-time error: calling a declared function whose definition is not linked. Run-time/input error: a user enters letters instead of a number; validate scanf's return value.

Warnings from -Wall -Wextra -pedantic identify suspicious code such as mismatched format specifiers, unused variables, or non-standard constructs. Fix warnings instead of ignoring them.

Single-file versus multi-file build

For a multi-file design (input.c, tariff.c, report.c plus a shared header), compile each source separately with gcc -c and link the resulting object files. After a change to one .c file, normally only that file needs recompilation followed by relinking. In a single-file design, the whole source is recompiled after a change. Object files use additional disk space, but incremental builds can save time for larger projects. The final program still processes n bills in O(n) time; each individual bill calculation is O(1).

Smart-Meter Design Challenge

The original design should be modified, not discarded. Keep integer-paise arithmetic, input validation, modular meter-rollover handling, and centralized tariff definitions. Replace the simple monthly reading input with timestamped 15-minute interval records. Store or stream the interval readings, classify each interval using the time-of-day tariff table, and calculate each charge using the tariff effective at that time. Keep tariff rates in one configuration/table rather than copying constants throughout the program. Validate missing, duplicate, out-of-order, or implausible readings. If m intervals are processed in chronological order, processing can be O(m); streaming needs O(1) working space beyond the current interval and tariff data, while storing all intervals needs O(m) space.

Testing Checklist

Normal usage: previous 1000, current 1250 → 250 units and Rs 480.00 bill under the stated example tariff.

Rollover: previous 65530, current 5 → 11 units consumed.

Invalid input: enter letters at a numeric prompt → program displays an error and exits safely.

DEBUG check: compile once normally and once with -DDEBUG; intermediate values should appear only in the debug build.

Compile all programs with -Wall -Wextra -pedantic and resolve warnings.

Type / purpose	Specifier
int	%d
unsigned int	%u
long	%ld
long long	%lld
float (printf)	%f
double (scanf)	%lf
char	%c
string (char array)	%s